LẬP TRÌNH JAVA CĂN BẢN · CHƯƠNG TỔNG HỢP: GIÁO TRÌNH VÀ THỰC HÀNH · BÀI TOÀN KHÓA

Lập Trình Java Căn Bản

Tài liệu gốc kèm thực hành bổ sung · 29/10/2026

Mục lục

- Cú Pháp Cơ Bản Java: Cài đặt JDK & IntelliJ
- Cú Pháp Cơ Bản Java: Biến, kiểu dữ liệu trong Java
- Cú Pháp Cơ Bản Java: Cấu trúc điều khiển
- Lập Trình Hướng Đối Tượng Trong Java: Class, Object, Constructor
- Lập Trình Hướng Đối Tượng Trong Java: Kế thừa & Đa hình
- Lập Trình Hướng Đối Tượng Trong Java: Interface & Abstract Class
- Xử Lý Ngoại Lệ & Collection: Try-catch & Custom Exception
- Xử Lý Ngoại Lệ & Collection: ArrayList & HashMap
- Xử Lý Ngoại Lệ & Collection: Dự án: Quản lý sản phẩm

CHƯƠNG 1 · BÀI 1

1.1. Cài đặt JDK & IntelliJ

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Phân biệt JDK, JRE và JVM
> - Cài đặt JDK và IntelliJ IDEA
> - Viết, biên dịch và chạy chương trình Java đầu tiên

1. JDK, JRE và JVM

JVM (Java Virtual Machine) là máy ảo chạy bytecode của Java. Mã nguồn Java được biên dịch thành bytecode, sau đó JVM chạy bytecode trên từng hệ điều hành, nhờ đó chương trình Java chạy được ở nhiều nơi. JRE (Java Runtime Environment) gồm JVM và thư viện chuẩn, đủ để chạy chương trình. JDK (Java Development Kit) gồm JRE cùng trình biên dịch `javac` và các công cụ phát triển; muốn viết chương trình cần cài JDK.

2. Cài đặt môi trường

1. Tải và cài JDK bản LTS (ví dụ 17 hoặc 21).
2. Kiểm tra bằng lệnh `java -version` và `javac -version` trong cửa sổ dòng lệnh.
3. Cài IntelliJ IDEA (bản Community là đủ) — môi trường phát triển tích hợp (IDE) phổ biến để viết mã Java.
4. Tạo dự án mới: File, New, Project, chọn JDK vừa cài.

3. Chương trình Java đầu tiên

Mỗi chương trình Java bắt đầu chạy từ phương thức `public static void main(String[] args)`. Tên file `.java` phải trùng với tên lớp public bên trong.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Xin chào Java!");
    }
}
```

Quy trình: `javac HelloWorld.java` biên dịch ra file `HelloWorld.class` (bytecode), rồi `java HelloWorld` chạy chương trình. Trong IDE chỉ cần bấm nút Run.

Tóm tắt

- JDK ⊃ JRE ⊃ JVM; viết chương trình cần JDK.
- Mã nguồn được biên dịch thành bytecode rồi chạy trên JVM.
- Điểm vào của chương trình là phương thức `main`.

Câu hỏi ôn tập và bài tập vận dụng

1. Cài JDK và IntelliJ, chụp lại kết quả lệnh `java -version`.
2. Viết chương trình in ra họ tên và lớp của bạn.
3. Giải thích vì sao Java được gọi là "viết một lần, chạy mọi nơi".

Cài đặt JDK & IntelliJ — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: IDE chạy được nhưng terminal báo không tìm thấy javac. JDK có công cụ biên dịch; kiểm tra `java -version`, `javac -version` và JDK cấu hình trong dự án. Tên file phải khớp lớp public theo cách tổ chức thông thường.

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Xin chào Java");
    }
}
```

Tự luyện: lưu Hello.java, chạy `javac Hello.java` rồi `java Hello`. **Hướng giải:** không thêm .class vào tên lớp khi chạy; phân biệt thư mục hiện tại và classpath. Không sửa PATH ngẫu nhiên trước khi xác định terminal đang dùng bộ JDK nào.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 1 · BÀI 2

1.2. Biến, kiểu dữ liệu trong Java

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Nắm các kiểu dữ liệu nguyên thuỷ và kiểu tham chiếu
> - Khai báo biến, hằng số và ép kiểu
> - Sử dụng String đúng cách

1. Kiểu dữ liệu nguyên thuỷ

Java có tám kiểu nguyên thuỷ: `byte`, `short`, `int`, `long`, `float`, `double`, `char` và `boolean`. Kiểu `int` lưu số nguyên 32 bit, `long` lưu số nguyên 64 bit (thêm hậu tố L), `double` là kiểu số thực mặc định, `char` lưu một ký tự Unicode và `boolean` chỉ nhận `true` hoặc `false`.

```
int tuoi = 20;
double diem = 8.5;
char xepLoai = 'A';
boolean dau = true;
long dan = 8_000_000_000L;
```

2. Biến và hằng số

Java là ngôn ngữ định kiểu tĩnh, mỗi biến phải được khai báo kiểu trước khi dùng. Từ khoá `final` dùng để khai báo hằng số, giá trị không thể gán lại: `final double PI = 3.14159;`.

3. Ép kiểu

Chuyển từ kiểu nhỏ sang lớn là tự động (`int` sang `double`); chuyển từ kiểu lớn sang nhỏ phải ép kiểu tường minh và có thể mất dữ liệu.

```
int a = 7;
double b = a;          // tự động: 7.0
int c = (int) 9.8;     // ép kiểu: 9
```

4. Kiểu tham chiếu và String

`String` là kiểu tham chiếu dùng để lưu chuỗi ký tự và là đối tượng bất biến (immutable). Để so sánh nội dung hai chuỗi phải dùng phương thức `equals`, không dùng toán tử `==`, vì `==` chỉ so sánh hai biến có cùng trỏ tới một đối tượng hay không.

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);       // false
System.out.println(a.equals(b));  // true
```

Tóm tắt

- Tám kiểu nguyên thuỷ; `double` là số thực mặc định.
- `final` tạo hằng số; ép kiểu từ lớn xuống nhỏ phải viết tường minh.
- So sánh chuỗi bằng `equals`, không dùng `==`.

Câu hỏi ôn tập và bài tập vận dụng

1. Khai báo biến cho các thông tin của một sinh viên với kiểu dữ liệu phù hợp.
2. Nhập hai số nguyên và in ra thương dưới dạng số thực.
3. Giải thích kết quả của `"abc" == "abc"` và `new String("abc") == new String("abc")`.

Biến, kiểu dữ liệu trong Java — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: đọc một số rồi đọc tên nhưng tên thành chuỗi rỗng. Khi dùng Scanner, nextInt để lại dấu xuống dòng; cách dễ kiểm soát là đọc nguyên dòng rồi chuyển đổi và kiểm tra lỗi.

```
int age = Integer.parseInt("20");
double average = (7 + 8) / 2.0;
System.out.println(age + 1); // 21
System.out.println(average); // 7.5
```

Tự luyện: chuyển `"hai mươi"`, chuỗi rỗng và số quá lớn. **Hướng giải:** parseInt có thể ném NumberFormatException; yêu cầu nhập lại chứ không tự biến dữ liệu sai thành 0. So nội dung String bằng equals hoặc Objects.equals khi cần xử lý null, không dựa vào == để so nội dung.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 1 · BÀI 3

1.3. Cấu trúc điều khiển

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Sử dụng câu lệnh rẽ nhánh if-else và switch-case
> - Sử dụng các vòng lặp for, while, do-while
> - Điều khiển vòng lặp bằng break và continue

1. Câu lệnh rẽ nhánh

Câu lệnh rẽ nhánh gồm `if`, `else if`, `else` và `switch`. Điều kiện của `if` phải là biểu thức `boolean`.

```
int diem = 8;
if (diem >= 8) {
    System.out.println("Giỏi");
} else if (diem >= 6) {
    System.out.println("Khá");
} else {
    System.out.println("Trung bình");
}
```

2. switch-case

Câu lệnh `switch` chọn một nhánh theo giá trị của biểu thức, mỗi nhánh thường kết thúc bằng `break` để không chạy tiếp sang nhánh kế. Nhánh `default` xử lý các giá trị còn lại.

```
switch (thu) {
    case 2: System.out.println("Thứ hai"); break;
    case 3: System.out.println("Thứ ba"); break;
    default: System.out.println("Khác");
}
```

3. Vòng lặp

Vòng lặp `for` dùng khi biết trước số lần lặp; `while` lặp khi điều kiện còn đúng; `do-while` luôn thực hiện thân vòng lặp ít nhất một lần vì kiểm tra điều kiện sau.

```
for (int i = 1; i <= 5; i++) {
    System.out.println(i);
}
int n = 10;
do {
    n--;
} while (n > 5);
```

4. break, continue và mảng

Lệnh `break` thoát khỏi vòng lặp, lệnh `continue` bỏ qua phần còn lại của lần lặp hiện tại. Mảng là cấu trúc lưu nhiều phần tử cùng kiểu với kích thước cố định, chỉ số bắt đầu từ 0; duyệt mảng bằng `for` hoặc `for-each`: `for (int x : mang) {...}`.

Tóm tắt

- `if`/`else`/`switch` để rẽ nhánh; nhớ `break` trong `switch`.
- `for`, `while`, `do-while` cho vòng lặp; `do-while` chạy ít nhất một lần.
- Mảng có kích thước cố định, chỉ số bắt đầu từ 0.

Câu hỏi ôn tập và bài tập vận dụng

1. Nhập điểm và in xếp loại bằng if-else.
2. In bảng cửu chương của một số nhập vào bằng vòng lặp for.
3. Tìm giá trị lớn nhất trong một mảng số nguyên.

Cấu trúc điều khiển — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: vòng lặp vượt giới hạn mảng. Với mảng n phần tử, chỉ số từ 0 tới n-1; điều kiện thường là i < values.length.

```
int[] values = {2, 4, 6};
int sum = 0;
for (int value : values) {
    sum += value;
}
System.out.println(sum); // 12
```

Tự luyện: mảng rỗng, một phần tử và ba phần tử. **Hướng giải:** tổng bắt đầu 0, không truy cập phần tử đầu khi chưa kiểm tra rỗng. Với while, chỉ ra biến làm điều kiện thay đổi; tránh nhầm phép gán và so sánh hoặc bỏ quên nhánh kết thúc.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 2 · BÀI 1

2.1. Class, Object, Constructor

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu khái niệm lớp, đối tượng và sự khác nhau giữa chúng
> - Viết lớp có thuộc tính, phương thức và constructor
> - Áp dụng tính đóng gói với getter và setter

1. Lớp và đối tượng

Lớp (class) là bản thiết kế mô tả thuộc tính và phương thức của một nhóm đối tượng. Đối tượng (object) là một thể hiện cụ thể của lớp, được tạo bằng từ khoá `new`. Thuộc tính lưu trạng thái của đối tượng, phương thức mô tả hành vi.

2. Constructor

Constructor là phương thức đặc biệt được gọi khi tạo đối tượng, có tên trùng với tên lớp và không có kiểu trả về, kể cả `void`. Nếu lớp không khai báo constructor nào, Java tự tạo constructor mặc định không tham số. Từ khoá `this` tham chiếu tới đối tượng hiện tại, thường dùng khi tên tham số trùng tên thuộc tính.

```
public class SinhVien {
    private String ten;
    private double diem;

    public SinhVien(String ten, double diem) {
        this.ten = ten;
        this.diem = diem;
    }
    public String getTen() { return ten; }
    public void setDiem(double diem) {
        if (diem >= 0 && diem <= 10) this.diem = diem;
    }
}
```

3. Tính đóng gói

Đóng gói (encapsulation) là việc che giấu dữ liệu bên trong lớp và chỉ cho truy cập qua phương thức. Thuộc tính thường được khai báo `private`, kèm phương thức getter và setter `public`; setter có thể kiểm tra dữ liệu hợp lệ. Các phạm vi truy cập gồm `private`, mặc định (không ghi), `protected` và `public`.

4. Thành phần static

Thành phần `static` thuộc về lớp chứ không thuộc về từng đối tượng, gọi được qua tên lớp, ví dụ `Math.sqrt(9)`. Dùng `static` cho biến đếm chung hoặc phương thức tiện ích.

Tóm tắt

- Lớp là bản thiết kế, đối tượng tạo bằng `new`.
- Constructor trùng tên lớp, không có kiểu trả về; `this` là đối tượng hiện tại.
- Đóng gói: thuộc tính `private`, truy cập qua getter và setter.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lớp HinhChuNhat với constructor, phương thức tính diện tích và chu vi.
2. Viết lớp TaiKhoan có phương thức rút tiền không cho rút quá số dư.
3. Tạo 3 đối tượng SinhVien và in thông tin ra màn hình.

Class, Object, Constructor — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: một sản phẩm được tạo với tên rỗng hoặc giá âm. Constructor và phương thức cập nhật nên bảo vệ bất biến, không chỉ giao diện nhập liệu.

```
final class Product {
    private final String id;
    private final String name;
    Product(String id, String name) {
        if (id == null || id.isBlank() || name == null || name.isBlank()) {
            throw new IllegalArgumentException("Thiếu mã hoặc tên");
        }
        this.id = id;
        this.name = name;
    }
}
```

Tự luyện: thử tên null, khoảng trắng và tên hợp lệ. **Hướng giải:** trạng thái không hợp lệ bị từ chối ngay, không tạo đối tượng nửa hợp lệ rồi sửa sau. this chỉ đối tượng hiện tại; constructor không có kiểu trả về.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 2 · BÀI 2

2.2. Kế thừa & Đa hình

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu và áp dụng kế thừa (inheritance)
> - Phân biệt nạp chồng (overloading) và ghi đè (overriding)
> - Hiểu đa hình (polymorphism) trong Java

1. Kế thừa

Kế thừa (inheritance) cho phép lớp con dùng lại thuộc tính và phương thức của lớp cha. Java dùng từ khoá `extends` và chỉ hỗ trợ đơn kế thừa với lớp. Từ khoá `super` dùng để gọi constructor hoặc phương thức của lớp cha. Mọi lớp trong Java đều kế thừa ngầm định từ lớp `Object`.

```
class DongVat {
    protected String ten;
    DongVat(String ten) { this.ten = ten; }
    void keu() { System.out.println("..."); }
}
class Cho extends DongVat {
    Cho(String ten) { super(ten); }
    @Override
    void keu() { System.out.println(ten + ": gâu gâu"); }
}
```

2. Phân biệt overloading và overriding

Nạp chồng (overloading) là khai báo nhiều phương thức cùng tên trong một lớp nhưng khác danh sách tham số. Ghi đè (overriding) là lớp con định nghĩa lại phương thức của lớp cha với cùng tên và cùng tham số. Overloading được xác định lúc biên dịch, overriding được xác định lúc chạy; chú thích `@Override` giúp trình biên dịch kiểm tra ghi đè.

3. Đa hình

Đa hình (polymorphism) là khả năng một lời gọi phương thức cho ra hành vi khác nhau tuỳ đối tượng thực tế. Ví dụ biến kiểu `DongVat` trỏ tới đối tượng `Cho` thì lời gọi `keu()` chạy phương thức của `Cho`.

```
DongVat[] ds = { new Cho("Mực"), new Meo("Mướp") };
for (DongVat d : ds) {
    d.keu();   // mỗi con kêu theo cách riêng
}
```

4. Từ khoá final

`final` đặt trước lớp thì lớp không thể bị kế thừa, đặt trước phương thức thì không thể bị ghi đè.

Tóm tắt

- `extends` để kế thừa; `super` gọi lớp cha; Java chỉ đơn kế thừa với lớp.
- Overloading: cùng tên khác tham số; overriding: lớp con định nghĩa lại phương thức lớp cha.
- Đa hình: hành vi phụ thuộc đối tượng thực tế lúc chạy.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết lớp Hinh với phương thức dienTich() và hai lớp con HinhTron, HinhChuNhat ghi đè phương thức này.
2. Viết ví dụ nạp chồng phương thức `tong` cho hai và ba tham số.
3. Giải thích sự khác nhau giữa overloading và overriding.

Kế thừa & Đa hình — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: nhiều loại sản phẩm có cách tính phí khác nhau. Lời gọi phương thức được chọn theo loại đối tượng thực khi có ghi đè.

```
class Item { double fee() { return 10; } }
class DigitalItem extends Item {
    @Override double fee() { return 0; }
}
Item item = new DigitalItem();
System.out.println(item.fee()); // 0.0
```

Tự luyện: cho nhiều Item vào danh sách và cộng fee. **Hướng giải:** dùng ghi đè thay vì kiểm tra chuỗi tên lớp. Overload là nhiều chữ ký tham số, khác override. Không ép kiểu xuống nếu chưa biết đối tượng thật; ưu tiên hợp đồng ở lớp cha.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 2 · BÀI 3

2.3. Interface & Abstract Class

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

Mục tiêu bài học

- Hiểu và viết lớp trừu tượng (abstract class)
- Hiểu và cài đặt interface
- Phân biệt interface và abstract class

1. Lớp trừu tượng

Lớp trừu tượng (abstract class) là lớp không thể tạo đối tượng trực tiếp, khai báo bằng từ khoá `abstract`. Lớp trừu tượng có thể chứa cả phương thức trừu tượng (không có thân) và phương thức thường có thân. Lớp con phải cài đặt mọi phương thức trừu tượng, trừ khi nó cũng là lớp trừu tượng.

```
abstract class Hinh {
    abstract double dienTich();
    void in() { System.out.println("Diện tích: " + dienTich()); }
}
class HinhTron extends Hinh {
    double r;
    HinhTron(double r) { this.r = r; }
    double dienTich() { return Math.PI * r * r; }
}
```

2. Interface

Interface là bản hợp đồng khai báo các phương thức mà lớp cài đặt phải thực hiện. Một lớp dùng từ khoá `implements` để cài đặt interface và có thể cài đặt nhiều interface cùng lúc. Từ Java 8, interface có thể chứa phương thức `default` có sẵn phần thân.

```
interface CoTheBay {
    void bay();
}
class Chim implements CoTheBay {
```

```
    public void bay() { System.out.println("Chim bay"); }
}
```

3. Phân biệt interface và abstract class

Một lớp chỉ kế thừa (`extends`) được một abstract class nhưng cài đặt (`implements`) được nhiều interface. Abstract class có thể có thuộc tính thường và constructor, interface chỉ có hằng số và không có constructor. Dùng abstract class khi các lớp có chung phần cài đặt, dùng interface khi chỉ cần chung khả năng.

4. Lợi ích

Cả hai giúp lập trình theo hợp đồng, làm code dễ mở rộng và dễ thay thế thành phần (ví dụ thay lớp lưu dữ liệu bằng file hoặc cơ sở dữ liệu mà không đổi nơi sử dụng).

Tóm tắt

- Abstract class: không tạo đối tượng trực tiếp, có thể có phương thức trừu tượng và thường.
- Interface: hợp đồng chỉ khai báo hành vi; lớp cài đặt nhiều interface bằng `implements`.
- Chọn abstract class khi có phần cài đặt chung, chọn interface khi cần chung khả năng.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết abstract class NhanVien với phương thức trừu tượng tinhLuong() và hai lớp con.
2. Viết interface CoTheDiChuyen và hai lớp cài đặt nó.
3. Nêu ba điểm khác nhau giữa interface và abstract class.

Interface & Abstract Class — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: cần thay nơi lưu dữ liệu nhưng giữ nguyên nghiệp vụ. Interface mô tả hợp đồng; abstract class có thể chia sẻ trạng thái và phần triển khai khi phù hợp.

```
interface PricePolicy { double price(double original); }
class HalfPrice implements PricePolicy {
    public double price(double original) { return original / 2; }
}
```

Tự luyện: thêm chính sách không giảm và kiểm thử qua biến PricePolicy. **Hướng giải:** nơi dùng không phụ thuộc chi tiết một lớp cụ thể; phương thức triển khai interface phải có mức truy cập phù hợp. Không chọn kế thừa chỉ để tránh viết vài dòng, nếu quan hệ đúng hơn là “sử dụng”.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 3 · BÀI 1

3.1. Try-catch & Custom Exception

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu ngoại lệ (exception) và phân loại ngoại lệ
> - Sử dụng try-catch-finally, throw và throws
> - Viết ngoại lệ tự định nghĩa

1. Ngoại lệ là gì

Ngoại lệ (exception) là sự kiện bất thường xảy ra khi chương trình đang chạy, làm gián đoạn luồng thực thi. Checked exception bắt buộc phải xử lý hoặc khai báo `throws`, ví dụ `IOException`. Unchecked exception kế thừa từ `RuntimeException`, ví dụ `NullPointerException` và `ArithmeticException`.

2. try-catch-finally

Khối `try` chứa đoạn mã có thể phát sinh ngoại lệ, khối `catch` bắt và xử lý ngoại lệ đó; một khối `try` có thể đi kèm nhiều khối `catch` cho các loại ngoại lệ khác nhau. Khối `finally` dùng để chứa đoạn mã luôn được thực thi dù có ngoại lệ hay không, thường để đóng file hoặc đóng kết nối.

```
try {
    int kq = 10 / soChia;
    System.out.println(kq);
} catch (ArithmeticException e) {
    System.out.println("Không chia được cho 0");
} finally {
    System.out.println("Kết thúc");
}
```

3. throw và throws

Từ khoá `throw` dùng để ném một ngoại lệ, ví dụ `throw new IllegalArgumentException("Sai giá trị")`. Từ khoá `throws` dùng để khai báo các ngoại lệ mà phương thức có thể ném ra ở chữ ký phương thức.

4. Ngoại lệ tự định nghĩa

Custom exception là lớp ngoại lệ do lập trình viên tự định nghĩa để mô tả lỗi nghiệp vụ, thường kế thừa từ `Exception` hoặc `RuntimeException`.

```
class InvalidPriceException extends RuntimeException {
    public InvalidPriceException(String msg) { super(msg); }
}

void datGia(double gia) {
    if (gia < 0) throw new InvalidPriceException("Giá không được âm");
}
```

Tóm tắt

- Checked phải xử lý hoặc `throws`; unchecked kế thừa `RuntimeException`.
- `finally` luôn chạy, dùng để giải phóng tài nguyên.
- `throw` ném ngoại lệ, `throws` khai báo ngoại lệ; có thể tự định nghĩa ngoại lệ nghiệp vụ.

Câu hỏi ôn tập và bài tập vận dụng

1. Viết chương trình nhập hai số và xử lý ngoại lệ chia cho 0 bằng try-catch.
2. Viết ngoại lệ tự định nghĩa TuoiKhongHopLeException và ném ra khi tuổi âm.
3. Giải thích vì sao nên dùng `finally` khi làm việc với file.

Try-catch & Custom Exception — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: lỗi bị bỏ qua bằng catch rỗng khiến chương trình vẫn báo lưu thành công. Chỉ bắt lỗi tại nơi có thể xử lý, bổ sung ngữ cảnh và giữ nguyên nguyên nhân khi chuyển tiếp.

```
try {
    int quantity = Integer.parseInt("abc");
    System.out.println(quantity);
} catch (NumberFormatException error) {
    System.out.println("Số lượng phải là số nguyên");
}
```

Tự luyện: quy định lỗi nhập dữ liệu khác lỗi đọc file. **Hướng giải:** người dùng có thể sửa input, còn lỗi I/O cần thông báo phù hợp; không lộ thông tin nhạy cảm trong thông báo. Dùng try-with-resources cho tài nguyên AutoCloseable, không dựa vào việc đóng thủ công ở cuối đường chạy thành công.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 3 · BÀI 2

3.2. ArrayList & HashMap

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Hiểu Collection Framework và các interface List, Set, Map
> - Sử dụng ArrayList để lưu danh sách
> - Sử dụng HashMap để lưu cặp khoá - giá trị

1. Collection Framework

Collection Framework là tập hợp các interface và lớp dùng để lưu trữ và thao tác nhóm đối tượng. Các interface chính gồm `List` (có thứ tự, cho trùng), `Set` (không trùng lặp) và `Map` (cặp khoá - giá trị). Collection chỉ lưu đối tượng, nên kiểu nguyên thuỷ được tự động đóng gói, ví dụ `int` thành `Integer`.

2. ArrayList

`ArrayList` là lớp cài đặt `List` dựa trên mảng động, tự tăng kích thước khi thêm phần tử. Phương thức `add` thêm phần tử, `get` lấy phần tử theo chỉ số, `remove` xoá phần tử và `size` trả về số phần tử. Generic như `ArrayList<String>` giúp kiểm tra kiểu dữ liệu ngay lúc biên dịch.

```
ArrayList<String> ds = new ArrayList<>();
ds.add("An");
ds.add("Bình");
for (String ten : ds) {
    System.out.println(ten);
}
System.out.println(ds.size());   // 2
```

3. HashMap

`HashMap` là lớp cài đặt `Map` lưu dữ liệu theo cặp khoá - giá trị, mỗi khoá là duy nhất. Phương thức `put` thêm hoặc cập nhật giá trị, `get` lấy giá trị theo khoá, `containsKey` kiểm tra khoá có tồn tại. `HashMap` không đảm bảo thứ tự phần tử; cần giữ thứ tự thêm vào thì dùng `LinkedHashMap`.

```
HashMap<String, Double> diem = new HashMap<>();
diem.put("An", 8.5);
diem.put("Bình", 7.0);
System.out.println(diem.get("An"));          // 8.5
for (String ten : diem.keySet()) {
    System.out.println(ten + ": " + diem.get(ten));
}
```

4. Chọn cấu trúc phù hợp

Dùng `ArrayList` khi cần danh sách có thứ tự và truy cập theo chỉ số; `HashSet` khi cần loại bỏ phần tử trùng; `HashMap` khi cần tra cứu nhanh theo khoá.

Tóm tắt

- `List` có thứ tự, `Set` không trùng, `Map` lưu cặp khoá - giá trị.
- `ArrayList` là mảng động; `HashMap` tra cứu nhanh theo khoá duy nhất.
- Dùng generic để kiểm tra kiểu ngay khi biên dịch.

Câu hỏi ôn tập và bài tập vận dụng

1. Nhập danh sách tên học viên vào ArrayList rồi in ra theo thứ tự.
2. Dùng HashMap lưu điểm của học viên và tìm điểm theo tên.
3. Loại các phần tử trùng trong một danh sách bằng HashSet.

ArrayList & HashMap — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: hai sản phẩm cùng mã làm dữ liệu cũ bị ghi đè. Map.put có thể thay giá trị hiện có; muốn từ chối trùng cần kiểm tra hoặc dùng thao tác phù hợp.

```
java.util.Map<String, String> products = new java.util.HashMap<>();
products.put("P01", "Bút");
if (products.putIfAbsent("P01", "Vở") != null) {
    System.out.println("Mã đã tồn tại");
}
```

Tự luyện: thêm, tìm mã không tồn tại và xóa khi duyệt danh sách. **Hướng giải:** không mặc định thứ tự HashMap; dùng iterator hoặc tạo danh sách mới khi cần lọc. Nhắc rõ ví dụ Map đơn luồng không tự giải quyết cạnh tranh ghi từ nhiều luồng.

Nguồn học tiếp

https://dev.java/learn/

CHƯƠNG 3 · BÀI 3

3.3. Dự án: Quản lý sản phẩm

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Xây dựng ứng dụng console quản lý sản phẩm bằng Java
> - Kết hợp lớp, ArrayList, HashMap và xử lý ngoại lệ
> - Rèn kỹ năng thiết kế chương trình theo lớp

1. Yêu cầu

Ứng dụng quản lý sản phẩm với các chức năng: thêm sản phẩm, hiển thị danh sách, tìm theo mã, cập nhật giá, xoá sản phẩm và thống kê tổng giá trị kho.

2. Thiết kế

- Lớp `Product` gồm các thuộc tính `id`, `name`, `price` cùng getter và setter; setter của `price` ném `InvalidPriceException` khi giá nhỏ hơn 0.
- Lớp `ProductManager` dùng `ArrayList<Product>` để lưu danh sách và `HashMap<String, Product>` để tra cứu nhanh theo mã.
- Lớp `Main` hiển thị menu, đọc lựa chọn bằng `Scanner` và gọi các phương thức tương ứng.

3. Cài đặt mẫu

```
class ProductManager {
    private final ArrayList<Product> ds = new ArrayList<>();
    private final HashMap<String, Product> theoMa = new HashMap<>();

    void them(Product p) {
        if (theoMa.containsKey(p.getId()))
            throw new IllegalArgumentException("Trùng mã sản phẩm");
        ds.add(p);
        theoMa.put(p.getId(), p);
    }

    Product tim(String id) { return theoMa.get(id); }
```

```
    double tongGiaTri() {
        double tong = 0;
        for (Product p : ds) tong += p.getPrice();
        return tong;
    }
}
```

4. Gợi ý mở rộng

Sắp xếp danh sách theo giá bằng `Comparator`; lưu danh sách ra file; thêm kiểu sản phẩm kế thừa (hàng điện tử, thực phẩm) để thực hành đa hình.

Tóm tắt

- Mỗi lớp đảm nhận một trách nhiệm rõ ràng (dữ liệu, xử lý, giao diện).
- Kết hợp ArrayList cho danh sách và HashMap cho tra cứu nhanh.
- Dùng ngoại lệ để báo dữ liệu không hợp lệ.

Câu hỏi ôn tập và bài tập vận dụng

1. Hoàn thiện ứng dụng với menu đầy đủ các chức năng.
2. Sắp xếp danh sách sản phẩm theo giá giảm dần.
3. Thêm chức năng ghi và đọc danh sách sản phẩm từ file.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập Lập Trình Java Căn Bản · Nội dung biên soạn trong dự án StudyOnline.

Java — Học thêm về thiết kế đối tượng và quản lý sản phẩm

Tài liệu nối tiếp các bài cú pháp, lớp, đa hình, ngoại lệ và collection. Ví dụ dùng cú pháp Java phổ biến, không yêu cầu preview feature.

Mục tiêu học thêm

- Phân biệt so sánh tham chiếu và so sánh nội dung
- Chọn collection theo thao tác cần thực hiện
- Kiểm tra dữ liệu trước khi cập nhật đối tượng và xử lý ngoại lệ có ý nghĩa

1. Tham chiếu và giá trị

Với đối tượng, == so sánh tham chiếu, còn equals thường dùng để so sánh nội dung theo cách lớp định nghĩa. Với String, dùng equals thay vì dựa vào việc các chuỗi có thể dùng chung vùng nhớ.

```
String a = new String("SV01");
String b = new String("SV01");
System.out.println(a == b);      // false
System.out.println(a.equals(b)); // true
```

Nếu biến có thể null, dùng Objects.equals(a, b) hoặc gọi equals từ chuỗi hằng. Với lớp tự viết dùng làm khóa HashMap, equals và hashCode phải nhất quán: hai đối tượng bằng nhau phải có cùng hashCode. Không thay đổi trường dùng làm khóa khi đối tượng đang nằm trong map.

2. Đóng gói và điều kiện hợp lệ

Đóng gói không chỉ là chuyển trường sang private rồi tạo setter. Setter phải bảo vệ điều kiện hợp lệ của đối tượng; nếu cập nhật lỗi, giá trị cũ phải còn nguyên.

```
class Product {
    private final String id;
    private int stock;
```

```
    Product(String id, int stock) {
        if (id == null || id.isBlank()) {
            throw new IllegalArgumentException("Mã không được rỗng");
        }
        this.id = id;
        setStock(stock);
    }

    void setStock(int newStock) {
        if (newStock < 0) {
            throw new IllegalArgumentException("Tồn kho không âm");
        }
        stock = newStock;
    }

    int getStock() { return stock; }
}
```

Mã sản phẩm không thay đổi sau khi tạo. Tồn kho được kiểm tra trước khi gán. Khi quản lý tiền, không tùy tiện dùng double cho các phép tính cần chính xác theo đơn vị tiền; có thể dùng số nguyên theo đơn vị nhỏ nhất hoặc BigDecimal với quy tắc làm tròn rõ ràng.

3. Interface, kế thừa và composition

Interface mô tả hợp đồng hành vi, ví dụ Repository có findById và save. Một triển khai lưu bằng bộ nhớ, một triển khai lưu bằng file có thể dùng cùng hợp đồng. Lớp dịch vụ không cần biết chi tiết lưu trữ.

Kế thừa phù hợp với quan hệ “là một” và lớp con giữ được kỳ vọng của lớp cha. Nếu chỉ cần dùng chức năng của một đối tượng khác, đưa đối tượng đó vào trường của lớp thường dễ bảo trì hơn kế thừa. Đa hình giúp gọi cùng phương thức qua interface nhưng mỗi triển khai xử lý khác nhau.

4. Collection và ngoại lệ

ArrayList phù hợp cho duyệt theo thứ tự và truy cập theo chỉ số; thêm/xóa ở giữa có thể cần dịch chuyển nhiều phần tử. HashMap phù hợp cho tìm theo mã với chi phí trung bình thấp nhưng không bảo đảm thứ tự duyệt. LinkedHashMap hữu ích khi cần giữ thứ tự chèn.

```
Map<String, Integer> stocks = new HashMap<>();
stocks.put("SP01", 10);
if (stocks.containsKey("SP01")) {
    int current = stocks.get("SP01");
```

```
    if (current >= 3) stocks.put("SP01", current - 3);
}
```

Đây là ví dụ đơn luồng; thao tác kiểm tra rồi cập nhật không tự an toàn khi có nhiều luồng. Không catch Exception rồi bỏ qua: người dùng sẽ tưởng thao tác thành công. Với đọc file, try-with-resources giúp đóng tài nguyên ngay cả khi có lỗi.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: giải thích kết quả == và equals trong ví dụ chuỗi.
2. Trung bình: viết sell(id, quantity), từ chối số lượng không dương và bán quá tồn; mã không có phải báo lỗi rõ.
3. Khó: tách ProductRepository và ProductService, viết hai repository bộ nhớ và file; mô phỏng lỗi ghi file mà không báo bán hàng thành công.

Gợi ý lời giải và tự kiểm tra

1. Hai new tạo đối tượng khác; equals của String so nội dung.
2. Kiểm tra quantity > 0, tồn tại sản phẩm và stock >= quantity trước khi cập nhật. Thử 0, -1, đúng bằng tồn và lớn hơn tồn.
3. Service thực hiện quy tắc nghiệp vụ, repository lưu/đọc. Lỗi lưu phải được truyền hoặc chuyển thành lỗi nghiệp vụ có nguyên nhân; không nuốt ngoại lệ. Kiểm thử file mất, nội dung sai định dạng, mã trùng và đối tượng null. Khi tách trách nhiệm đúng, đổi cơ chế lưu không buộc viết lại quy tắc bán hàng.

Nguồn học tiếp

https://dev.java/learn/ ; mục Object Oriented Programming, Collections và Exceptions.

Dự án: Quản lý sản phẩm — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Bài thực hành: quản lý mã duy nhất, tên, giá và số lượng; hỗ trợ thêm, tìm, cập nhật và xuất danh sách. Chia lớp dữ liệu, dịch vụ và nhập/xuất. Dùng BigDecimal hoặc đơn vị tiền nguyên theo quy ước bài tập thay vì tùy tiện dùng double cho tiền thực.

Tự luyện: tạo 8 ca: tên rỗng, mã trùng, giá âm, số lượng 0, tìm không thấy, dữ liệu hợp lệ, file thiếu và file hỏng. **Hướng giải:** xác định quy tắc số lượng 0 trước, kiểm tra trạng thái sau thao tác bị từ chối phải không đổi. Ghi rõ phần chưa triển khai, không coi việc chương trình biên dịch là đã kiểm thử đầy đủ.

Nguồn học tiếp

https://dev.java/learn/